MULTI-AGENT REINFORCEMENT LEARNING · RESEARCH REPORT

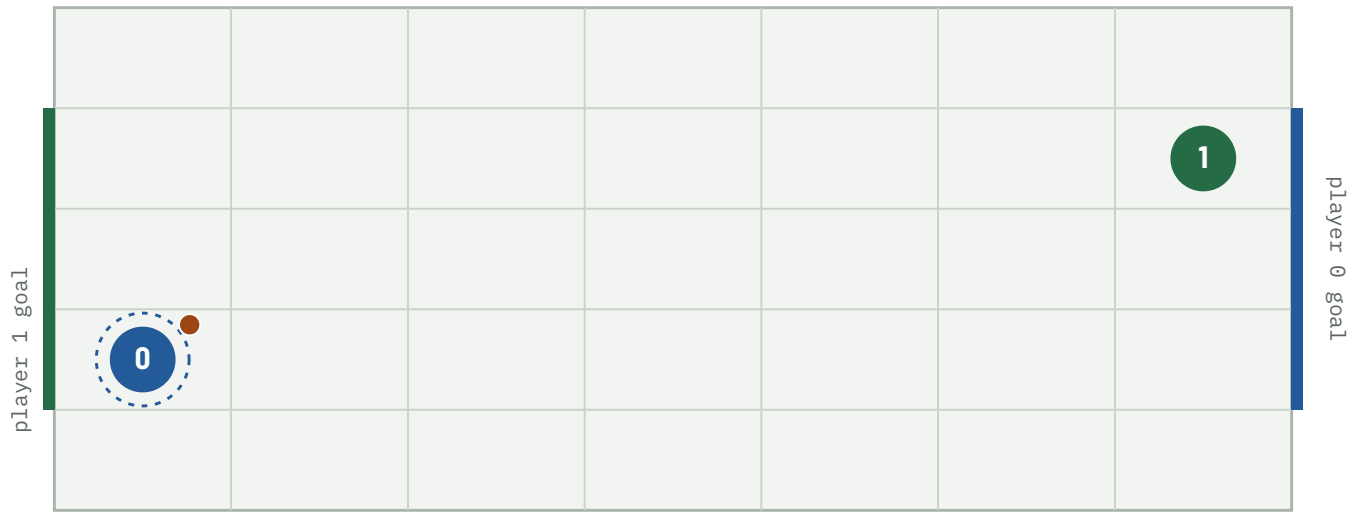
When Does Soccer Need Mixed Strategies?

A pure-first Nash-Q approach: where a two-player soccer Markov game does — and does not — need mixing, and how much LP work that saves.

FOUR QUESTIONS

RQ1 — when does the soccer Markov game admit pure versus mixed stage equilibria? **RQ2** — can pure-saddle detection reduce Nash-Q computation without changing the exact value? **RQ3** — what spatial and transition structures characterize the mixed-equilibrium states? **RQ4** — how sensitive are classification and computation to discounting, tolerance, and reward formulation?

repo · soccer-markov-nash | 210+ tests | game family in docs/design.md | methods in docs/methods.md | every policy drawn in docs/gallery.html



state = (x0, y0, x1, y1, b) = (0, 1, 6, 3, 0)

The kickoff. 7×5 grid, 2 380 non-terminal states. Player 0 (blue) carries the ball and attacks the right edge; player 1 (green) the left. Start positions are the ones the

A10 page assigns this netID. Actions `U D L R` are chosen simultaneously; reward is `+1 / −1 / 0`, zero-sum.

POSITION RELATIVE TO LITTMAN (1994)

Littman **already established** that the soccer Markov game can require mixed strategies — his Figure 2 gives a state where any deterministic offensive choice is blocked indefinitely and only randomising between `stand` and `move` guarantees an opening. This project does not rediscover that. It asks: **where does mixing appear, why does it appear, and what parameter switches it on?** Littman demonstrated the phenomenon in a *particular* soccer Markov game; we investigate which structural features determine when mixed equilibria arise, and whether their computational cost can be avoided except where mixing is genuinely necessary (`docs/littman.md` reproduces his Figure 2 and Table 3).

ABSTRACT

1 **The soccer game returns pure strategies until the collision rule couples the ball-steal to both players' actions.** Deterministic or coin-flip resolution → a pure saddle at every one of the 238 000 (state×step) stage games (also positionally determined). Littman's random *move order* → mixing, but only when it is the majority rule (sharp threshold at `blend ≈ 0.5` in this interpolation family) and only with a multi-cell goal. That last clause is specific to the move-order rule: action-independent movement noise (`slip`) forces mixing at any goal width.

---

2 **The pure-first hybrid eliminates LP work on almost every stage game** — 100% of states on the deterministic game, 96.05% on the random one — matching the all-LP value function to `4e−16`. That is a property of the game; the wall-clock speedup is reported separately.

---

3 **Structural characterization with a per-stage-game certificate:** the mixed states combine stochastic move order, a multi-cell goal mouth, and a defender with insufficient one-step coverage; *every* tested mixed stage game contains a 2×2 matching-pennies submatrix, and the strongly-mixed core is ~26 (entropy ≥ 0.9 bits). The single-cell → pure / wider → mixed switch is machine-certified invariant to the discount, the action set, and the reward objective (`make verify`); the region carries 41% of the equilibrium-path occupancy at 4% of the state space.

---

4 **Even a policy network that names the exact-optimal action 99.9% of the time is more exploitable than a mediocre value network** — "name the right action" is not "play a Nash". The exact solver stays the ground truth.

Method

CONTRIBUTIONS AND HYPOTHESIS

1. Algorithmic: a pure-first hybrid Nash-Q backup — check for a pure saddle ($O(A^2)$), use it, LP only where it fails; exact everywhere; eliminates LP work on 96–100% of stage games. **2. Empirical structural characterization** (over the tested parameter grid, not a theorem) of where the mixed region is and what switches it on.

Hypothesis: mixed equilibria are not distributed arbitrarily — they arise from a specific interaction between goal-mouth geometry and stochastic move-order coupling, and because most states keep pure saddles a pure-first solver gets the exact value while substantially cutting LP usage.

OBJECTIVE DEFINITIONS — RELATED BUT NOT INTERCHANGEABLE

WHERE	OBJECTIVE
A10 Part 1 / exact	undiscounted, finite 100-step horizon (`run_finite_horizon`, non-stationary V)
stationary Nash-Q	discounted infinite-horizon fixed point, $\gamma < 1$ (`run()`) — a contraction proxy for the above
`scoring="rate"`	continuing discounted: a goal resets, value = expected discounted goal difference
Littman replication	Littman's discount / draw interpretation, $\gamma = 0.9 \equiv 0.1$ per-step draw probability

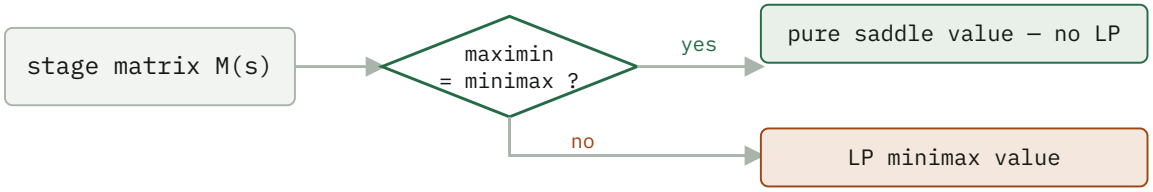
Unless a section says otherwise, results are the stationary discounted fixed point at $\gamma = 0.9$; RQ4 shows the pure/mixed classification is stable across $\gamma \in [0.5, 0.995]$ and matches the exact undiscounted answer.

The game is zero-sum, so at every state the stage game is a payoff matrix $M(s)[a_0, a_1] = E[R + \gamma V(s')]$ and the Shapley (1953) fixed point is the minimax value:

$$V(s) = \mathrm{val}(M(s)) = \max_p \min_k (p^T M)[a_1] = \min_p \max_{a0} (Mq)[a_0]$$

The $Q = R + \beta\,\mathrm{Nash}(Q')$ update is this operator. The A10 game is *undiscounted* (±1 at a goal, tie after 100 steps); $\gamma < 1$ is a contraction device for value iteration and RQ4 shows the split holds across γ. Three stage backups: **pure**

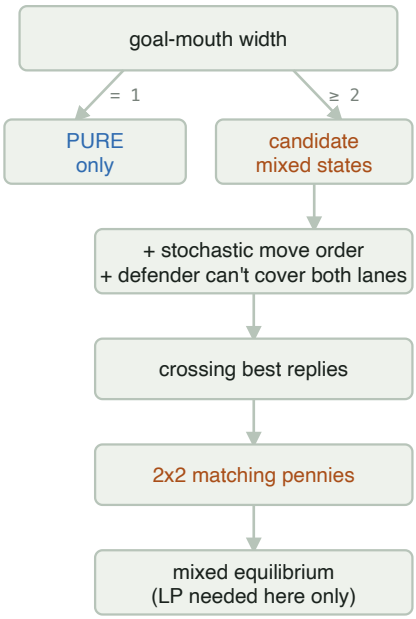
(the maximin security value — a fast lower bound, not a Nash value when it is below the minimax), **mixed** (the LP minimax value, via `scipy` HiGHS), and **hybrid** (the pure saddle value where `maximin = minimax`, the LP elsewhere — exact everywhere). Support enumeration (`support_enum.py`) finds *all* equilibria, including of general-sum games.

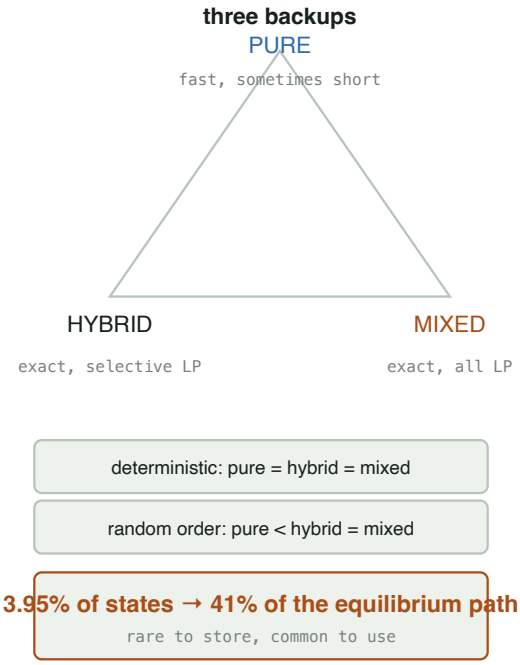


The pure-first hybrid. On the deterministic game the "yes" branch is taken at every one of the 2 380 states, so the LP is never called.

RESULTS

Where mixing comes from, and what the solver does about it





The whole story. Left: the mechanism chain — goal width, then stochastic move order and interception geometry, produce crossing best replies and a matching-pennies stage game where the LP is needed. Right: the three backups, the experimental result, and the occupancy headline.

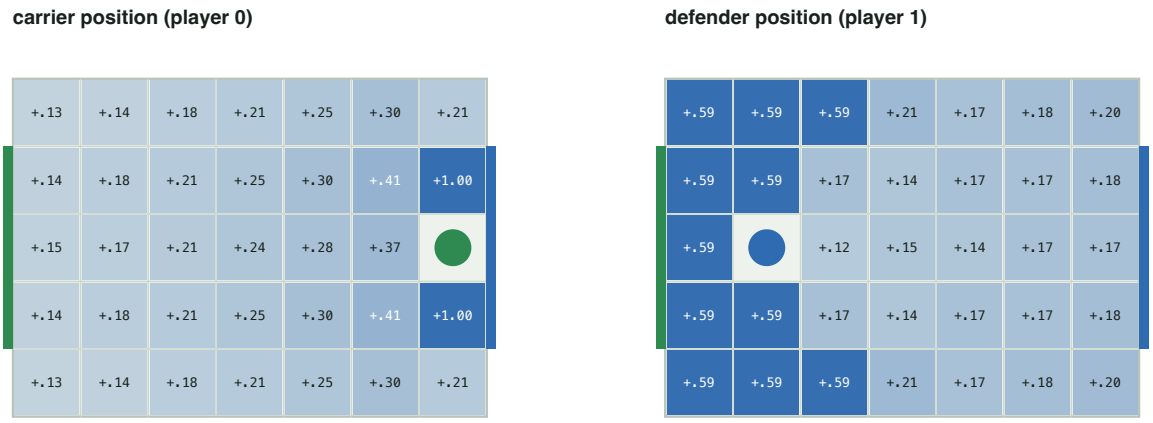
RQ1 — pure versus mixed, by collision rule

The A10 game is undiscounted, so it is solved exactly by 100-step backward induction (`run_finite_horizon`, `experiments/undiscounted.csv`):

EXACT UNDISCOUNTED GAME — 238 000 (STATE × STEP) STAGE GAMES

MOVE ORDER	MIXED STAGE GAMES	V(KICKOFF)	PURE EQUILIBRIUM?
deterministic (exact A10)	0	0.000	yes — pure (non-stationary)
coinflip tie-break	0	0.000	yes
random move order	3 148 (404 states)	+0.459	no

The deterministic equilibrium is **constructive and memoryless**: each player's win attractor (states from which it forces a goal, `soccer_nash/attractor.py`) equals its $V^* = \pm 1$ set, and the pure profile "attractor move on your winning set, safety move elsewhere" realizes V^* at every state (`scripts/positional.py`) — the game is positionally determined.



V* by player 0 position (blue = player 0 ahead)

V* by player 1 position (blue = player 0 ahead)

The value surface of the random-order game ($\gamma = 0.9$). Left: V^* by carrier position — a smooth gradient toward the attacking goal, saturating to +1 on the two cells that score in one move. Right: V^* by defender position with the carrier held near midfield — nearly flat, the defender's location barely moves the value.

The stationary $\gamma < 1$ solve agrees and is faster — 0 no-pure-saddle stage games on the deterministic game at every γ from 0.5 to 0.995, and **94** / **2380** on the random game at $\gamma = 0.9$ ($V(\text{kickoff}) = +0.164$ at the netID start). The no-pure-saddle count never depends on the kickoff.

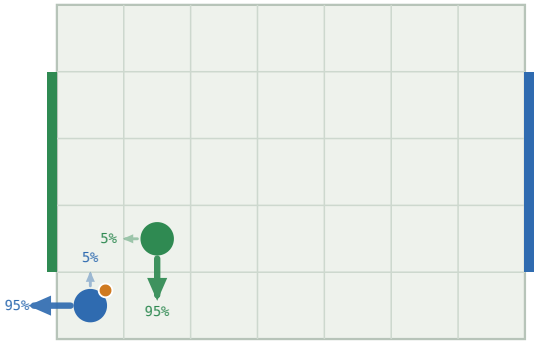
IT IS THE MOVE ORDER, NOT THE RANDOMNESS — AND IT HAS TO BE THE MAJORITY

The deterministic and coin-flip value functions are *identical* (`max |V_det − V_coin| = 0` at every γ): optimal play never enters a losing contest, so the coin is never flipped. Only Littman's random move order — where a ball-steal depends on who resolves first *and* on both players' targets — creates matching-pennies subgames. Interpolating the two (`move_order="blend"`, resolve by random order with probability `blend`) shows the onset is **sharp**: **0** mixed stage games for `blend ≤ 0.5`, dozens at `blend = 0.52`, and an *overshoot* to ~1.5× the fully-random count near `blend ≈ 0.7` before it relaxes (`scripts/blend.py`, `docs/blend.md`).

Scope of this claim: in this particular soccer family, this one coin-flip collision variant does not create new mixed stage games, whereas random move ordering does — and only when it is the majority rule in the `P = (1−p)P_det + pP_random` interpolation. This is *not* a statement about all action-independent stochastic transitions, which can certainly alter equilibrium structure in other games. Whether `p_c ≈ 0.5` is structural is open.

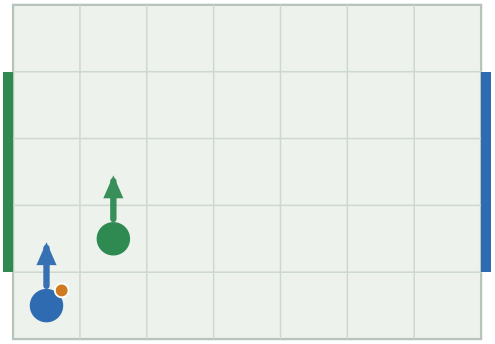
A threshold, not a ramp. No-pure-saddle stage games vs. the fraction of moves resolved by random order (5×4 board). Nothing until the random rule is the majority; then a jump and an overshoot.

random order -> mixed



mixed equilibrium · V = +0.078

deterministic -> pure



pure equilibrium · V = +0.000

One state, two resolution rules. Arrows are the equilibrium action distribution, sized by probability. Random order (Littman) makes the stage game matching pennies; deterministic carrier-wins collapses it to a pure best reply. Full set: docs/gallery.html.

RQ2 — Efficiency

Two numbers, kept apart: the **LP-call rate** is a property of the game and reproduces exactly; **wall clock** is a median of 5 repeats and depends on the machine and LP backend (`scripts/benchmark.py`).

RANDOM MOVE ORDER, Γ = 0.9 — THE CASE WHERE THE LP IS ACTUALLY NEEDED

SOLVER	SWEEPS	LP CALLS	STATES NEEDING LP	MEDIAN WALL CLOCK
pure (lower bound)	18	0	—	0.3 s
hybrid exact	108	9 672	3.95%	13.2 s
all-LP exact	108	~3.6 M	100%	~8 min

The portable claim: the hybrid eliminates LP work on almost every stage game — 100% of states on the deterministic game, **96.05%** on the random one — and matches the all-LP value to 4e−16. That is a property of the game, not the machine. The observed wall-clock ratio (~13 s vs ~8 min) is reported separately: it also reflects matrix construction and the LP backend. **Mirror symmetry** (`run_symmetric`, deterministic dynamics only): every state has a distinct board-flip-plus-player-swap mirror, so the 2 380 states are 1 190 pairs — reconstruct one from the other by `V(mirror(s)) = −V(s)` (and the equilibrium *policies* are equivariant too, where unique: `verify_policy_equivariance` returns 0 for the random game's 56 mixed states).

3.95% OF STATES, BUT 41% OF THE EQUILIBRIUM PATH

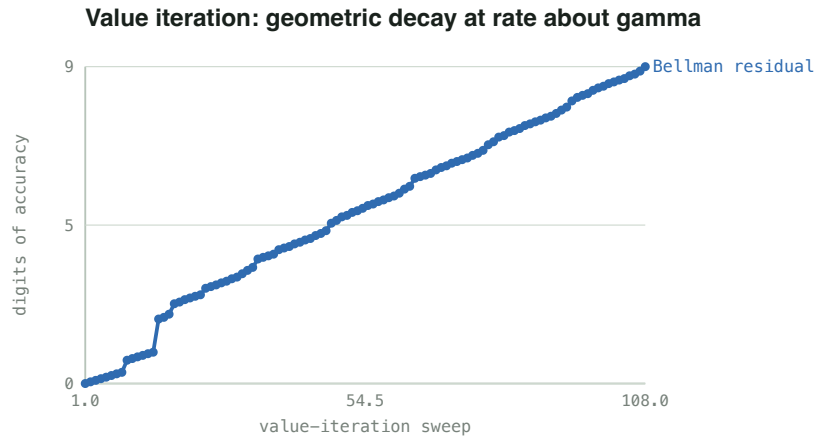
The LP-call rate counts the whole state space. Under the Nash policy from the kickoff only **456 of 2 380** states are ever reached, and the 94 no-pure-saddle states carry **41%** of the discounted occupancy — a 2× over-representation on the path, 10× vs. the raw count (`soccer_nash/occupancy.py`, `docs/occupancy.md`). The carrier's optimal march to goal runs straight through the contested band; four of the eight most-visited states are mixed. The mixed structure is rare to *store* and common to *use*.

Freeze-then-iterate, and its staleness

RANDOM MOVE ORDER, HYBRID SOLVER — SAME FIXED POINT, |DIFF| < 3E−9

	ROUNDS / SWEEPS	MATRIX-GAME SOLVES	WALL CLOCK
value iteration	108	9 672	13.8 s
policy iteration	21	1 879	17.2 s

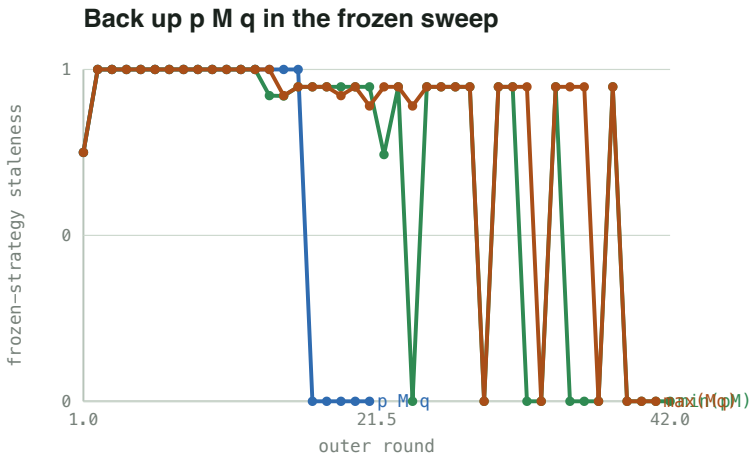
5× fewer LP solves — but the frozen strategies' distance to the current Nash stays *maximal* (a full pure-strategy flip) for **16 outer rounds**, then collapses to < 1e−8. The thrashing eats the saving; on the deterministic game it is strictly worse. Value iteration with the per-sweep Nash cache wins here.



Value iteration converges cleanly. The Bellman residual decays geometrically at rate ≈ γ — 108 sweeps to 1e−9. Freeze-then-iterate, by contrast, makes *no* progress until the frozen stage saddle flips (next figure) — concurrent stochastic games have no monotone policy improvement (docs/discussion.md §4).

WHICH VALUE QUANTITY TO BACK UP IN THE FROZEN SWEEP

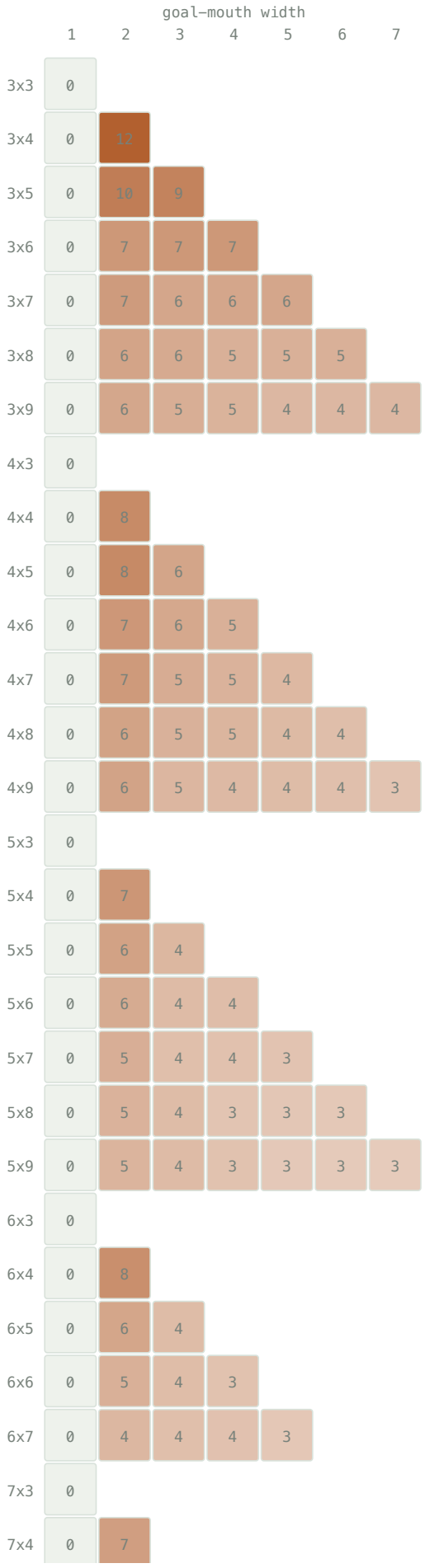
The professor's specific question: the frozen (p, q) don't match the current Q, so does the evaluation sweep back up $p^\top Mq$, $\min_k (pM)_k$, or $\max_i (Mq)_i$? All three reach the *same* fixed point (zero-sum equilibria are interchangeable), but **$p^\top Mq$ converges in half the rounds** (21 vs 41–42) — it is the unbiased estimate while the strategies are stale, where the two bounds are systematically pessimistic / optimistic and stretch the thrash phase from 16 rounds to 28 (run_policy_iteration(eval_value=...), docs/numerics.md §3).

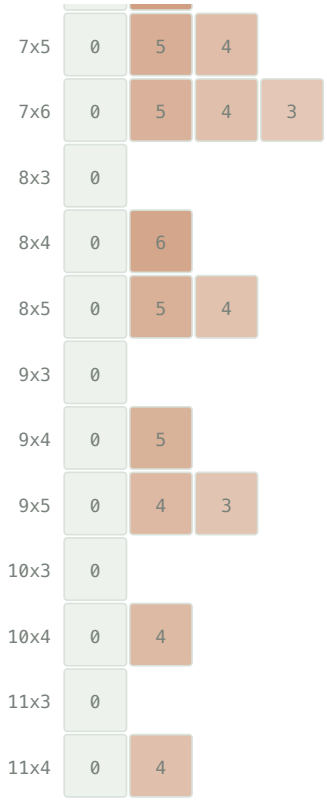


Back up $p^{\top}Mq$. Staleness of the frozen strategies per outer round. The middle quantity settles by round 16; the pessimistic $\min_k (pM)_k$ and optimistic $\max_i (Mq)_i$ keep flipping until round 28.

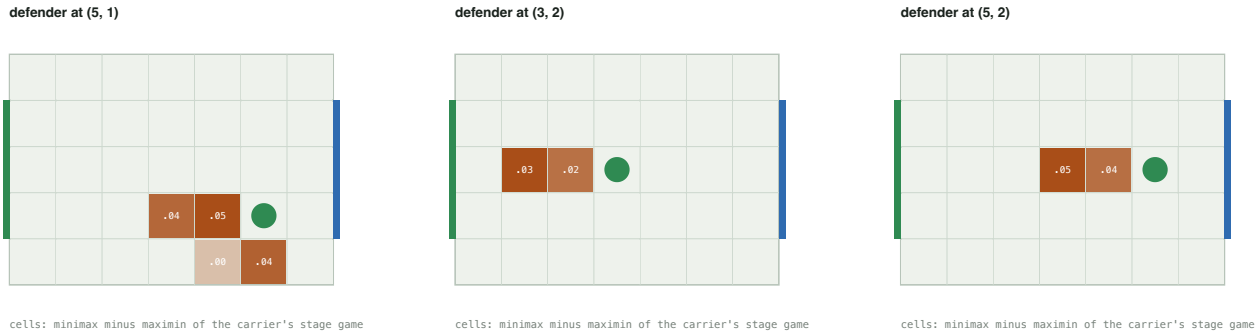
RQ3 — what characterizes the mixed states

Across the tested family, a one-cell goal produced no mixed stage games while every tested goal width ≥ 2 produced some. The phase diagram sweeps **127 configurations** — boards with `3 ≤ width ≤ 11`, `3 ≤ height ≤ 9`, `width·height ≤ 45`, against goal widths `1` to `height−2` (`scripts/phase_diagram.py --analyze`). Precisely: a multi-cell goal is *necessary* for the mixed region in the studied family, and across the tested boards *sufficient* for at least one mixed state — not a proof that every such board must have one, nor that every state is mixed.


goal–mouth width
1 2 3 4 5 6 7
3x3 0
3x4 0 12
3x5 0 10 9
3x6 0 7 7 7
3x7 0 7 6 6 6
3x8 0 6 6 5 5 5
3x9 0 6 5 5 4 4 4
4x3 0
4x4 0 8
4x5 0 8 6
4x6 0 7 6 5
4x7 0 7 5 5 4
4x8 0 6 5 5 4 4
4x9 0 6 5 4 4 4 3
5x3 0
5x4 0 7
5x5 0 6 4
5x6 0 6 4 4
5x7 0 5 4 4 3
5x8 0 5 4 3 3 3
5x9 0 5 4 3 3 3 3
6x3 0
6x4 0 8
6x5 0 6 4
6x6 0 5 4 3
6x7 0 4 4 4 3
7x3 0
7x4 0 7



The gate (127 configs, 3≤W≤11, 3≤H≤9, W·H≤45). Goal width 1 → **0** mixed stage games on all 40 tested one-cell configs. Goal width ≥ 2 → at least one on all 87 tested wider-goal configs. The *fraction* (2.7–12%) is an OLS R^2 0.64 in board area — a dilution effect — goal width adding little with a negative coefficient.



Where the carrier has to guess. Defender pinned at the green disc; each cell shaded by `minimax − maximin` of the carrier's stage game (0 = pure saddle). Mixing is a local phenomenon — a handful of cells where the defender contests the forward lane.

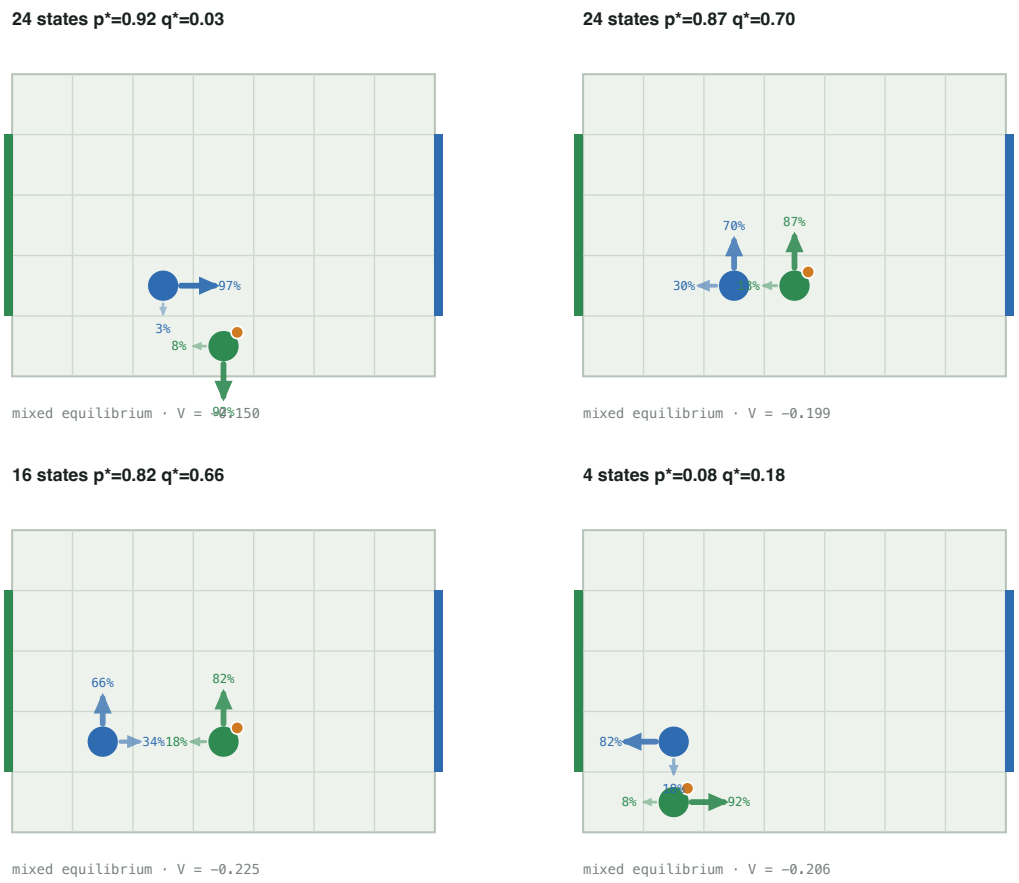
Geometry predicts the label (`scripts/geometry_model.py`): a depth-4 decision tree on carrier-frame features separates `mixed` from the rest with **precision 0.96, recall 0.91**. The dominant feature is `defender_can_intercept` — the defender within one move of the carrier's forward cell (`P(mixed) = 0.44` vs `0.002`).

Beyond prediction, the 94 mixed states canonicalize under the board mirror to 47 pairs and cluster into **8 geometric templates** (`scripts/templates.py`, `docs/templates.md`).

THE MIXED-STATE TEMPLATES — CARRIER-FRAME GEOMETRY, EQUILIBRIUM SUPPORT

STATES	GEOMETRY	SUPPORT	STRUCTURE
24	defender 1 ahead, carrier one row off the goal row	2×2	matching pennies
24	defender 1 ahead, same row, carrier on a goal row	2×2	matching pennies
16	defender 2 ahead, same row, carrier on a goal row	2×2	matching pennies
4	defender diagonally adjacent, carrier off the goal row	2×2	matching pennies
14	defender 1–2 ahead, same row, on a goal row	2×1	near-pure saddle
8	defender 2 ahead, same row, on a goal row	3×2	degenerate (row ties)
4	defender 2 ahead, carrier off the goal row	3×3	wider mix

The **four 2×2-support templates are all verified matching pennies** — the carrier's best reply flips between the two defender columns and vice versa — covering **68 of 94** states: carrier {climb, advance} against defender {cover a lane, hold the forward cell}. All 94 share one value across every equilibrium; 64 have a unique one.



One policy fan per matching-pennies template. Each covers a class of mixed states related by carrier-frame geometry; p*, q* are the equilibrium weights on the first support action. The carrier randomizes between two scoring lanes, the defender between covering them.

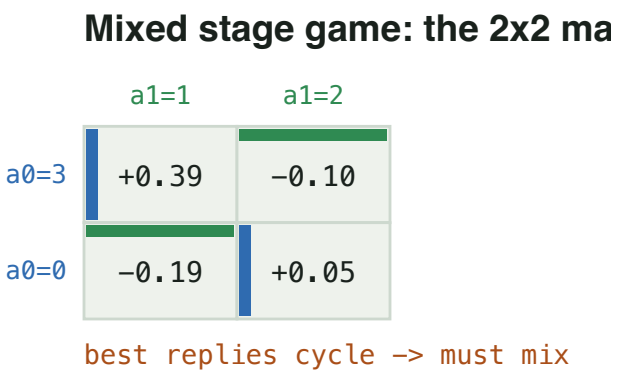
The mechanism (machine-certified per config; interpretable, not a general theorem): a stage game requires mixing when a **stochastic** (½–½) **resolution order** meets a **multi-cell goal** and a **defender that can contest the forward cell but not cover both lanes**. Then the carrier's "which lane" reply flips with the defender's cover and vice versa — *crossing best replies*, hence no pure saddle:

```
geometric condition ⇒ crossing best replies ⇒ no pure saddle
```

Drop any one clause — a one-cell goal, deterministic resolution, the coin-flip tie-break — and every tested stage game has a pure saddle. `docs/mechanism.md` gives the interpretable argument for the single-cell case. What is *not* settled is a board-size-free theorem for arbitrary `W, H`. *Supporting theory only* (per the meeting feedback, not a deliverable): for a single goal cell the defender's optimal strategy is closed-form and machine-verified, and every single-cell stage game is dominance-solvable, checked for all boards up to 11×5.

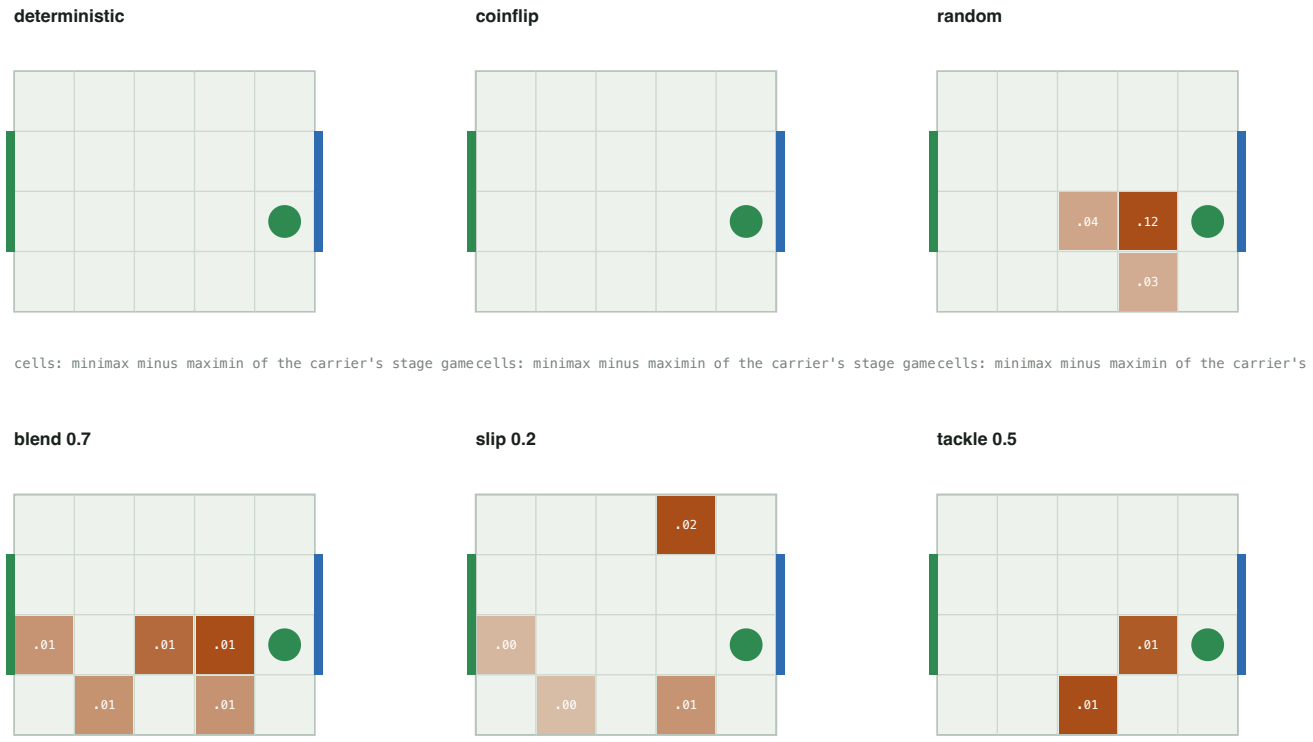
A PER-STAGE-GAME CERTIFICATE

`soccer_nash/certificate.py` re-derives the pure/mixed label from the stage matrix with $O(A^2)$ arithmetic and no LP; `scripts/verify_mechanism.py` (`make verify`) runs it over the reachable state space. Single goal cell → a saddle-cell certificate at every state (0 mixed, cross-checked against `onecell.certify`). Wider goal → every mixed stage game gets a certificate an independent checker re-verifies to ~1e−16: the `maximin < minimax` gap (the LP-free proof that no pure saddle exists), a closed best-reply cycle, the exact equilibrium (ε < 1e−15), and a **2×2 matching-pennies submatrix — present in all 94 of the 7×5 mixed states and all 56 of the 5×4 ones**, strictly stronger than the 68-of-94 support count. The switch is checked invariant across $\gamma \in \{0.5, 0.9, 0.995\}$, 4 vs 5 actions, `win` vs `rate` scoring, and exact undiscounted backward induction (16/16), and a ±1e−6 perturbation flips no genuine classification. `docs/result.md` is the precise statement and the prior-work positioning.



The 2×2 matching-pennies core of a mixed stage game. Blue bar = player 0's best row per column; green bar = player 1's best column per row. They never coincide — the best replies cycle, so mixing is forced. Every tested mixed stage game contains one.

How far the goal-width switch generalizes (`scripts/generalize.py`, `docs/generalize.md`). It survives board scale (checked to 11×5 and 9×7), aspect ratio, and goal placement: non-contiguous goal cells like `(1, 3)` or `(0, 2, 4)` still produce mixed stage games, so adjacency does not matter, only that the carrier has two cells the defender cannot cover at once. It does *not* survive a change of transition family. Adding action-independent movement noise (`SoccerGame(slip=p)`: each player takes a random move with probability p) produces mixed stage games even under deterministic resolution and even with a single goal cell — a 5×5 one-cell board goes from 0 mixed to 62 at `slip = 0.1`. The same is true of this project's own `tackle` rule — the defender commits to a challenge that wins the ball with probability `tackle_prob` or bounces off (`docs/tackle.md`). So the switch is a property of Littman's move-order rule, where the unresolved coin only bites where the carrier has two lanes; `slip` and `tackle` supply that coin everywhere, a fair coin flip on contested squares supplies it nowhere on the equilibrium path. Reading the 2×2 cores of the 7×5 mixed states, the carrier's crossing pair is a vertical move in 90 of 94 — it is choosing which goal row to head for, and the defender is guessing it.



A fingerprint per collision rule. Same board, same pinned defender; shading is `minimax − maximin` of the carrier's stage game. Each rule for resolving a contested cell leaves its own mark: the deterministic and coin-flip rules stay pure everywhere, Littman's move order and its blend light a thin band at the goal mouth, and the two rules that put an unavoidable coin on every square — `slip` and `tackle` — light a broad region around the defender.

EXISTENCE IS NOT THE SAME AS AMOUNT

Every one of the 94 no-pure-saddle stage games sits within `0.07` of a pure saddle (`minimax − maximin`, median `0.029`), and the carrier's **mixing entropy** has median `0.55` bits — most are near-pure hedges (≈ 87/13), not real randomisation. Only **26 of 94** reach `≥ 0.9` bits (a near-even split); those are the states with genuine indifference. So the mixed region is 94 states, but the *strongly*-mixed core is ~26. Robust structure: 68 have a 2×2 matching-pennies support; the *value of mixing* (`V(hybrid) − V(pure maximin)`) averages `+0.13` there and compounds — at the centred kickoff a pure player secures only a draw where mixing is worth `+0.15` (`scripts/mixing.py`).

The reward objective does not move the mixed region

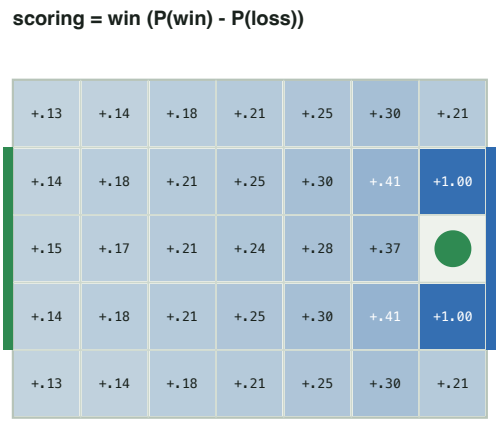
The default game rewards a *win* — first goal ends it, value `P(win) − P(loss)`. A second objective, `scoring="rate"` (`scripts/reward.py`), keeps playing: a goal scores `±1` and restarts with the ball to the conceding

team, so the value is the expected discounted *goal difference* and `γ < 1` is load-bearing. On the 7×5 random-order board:

SAME DYNAMICS, TWO REWARD OBJECTIVES — RANDOM MOVE ORDER, DISCOUNT 0.9

OBJECTIVE	NO-SADDLE STATES	VALUE RANGE	V(KICKOFF)
`win — P(win) - P(loss)`	94	[−1.00, +1.00]	+0.150
`rate — expected goal difference`	94 — the same 94	[−0.88, +0.88]	+0.132

The no-pure-saddle set is *identical* — **0** states enter or leave. A mixed Nash equilibrium is an equilibrium of a single stage game `M(s)`; changing how the horizon is scored rescales `M(s)` through `V(s')` but does not cross the `maximin = minimax` boundary. The value *range* compresses (`±1 → ±0.88`): under `rate` a lost position is not a cliff — conceding restarts play with possession, a floor under every state. Details in `docs/reward.md`.



V* by player 0 position (blue = player 0 ahead)

V* by player 0 position (blue = player 0 ahead)

Same shape, compressed range. `V*` by carrier position under the two objectives. `rate` pulls the goal-adjacent `+1` cells down to `+0.88` — the concede-and-restart floor — without changing where the value gradient runs.

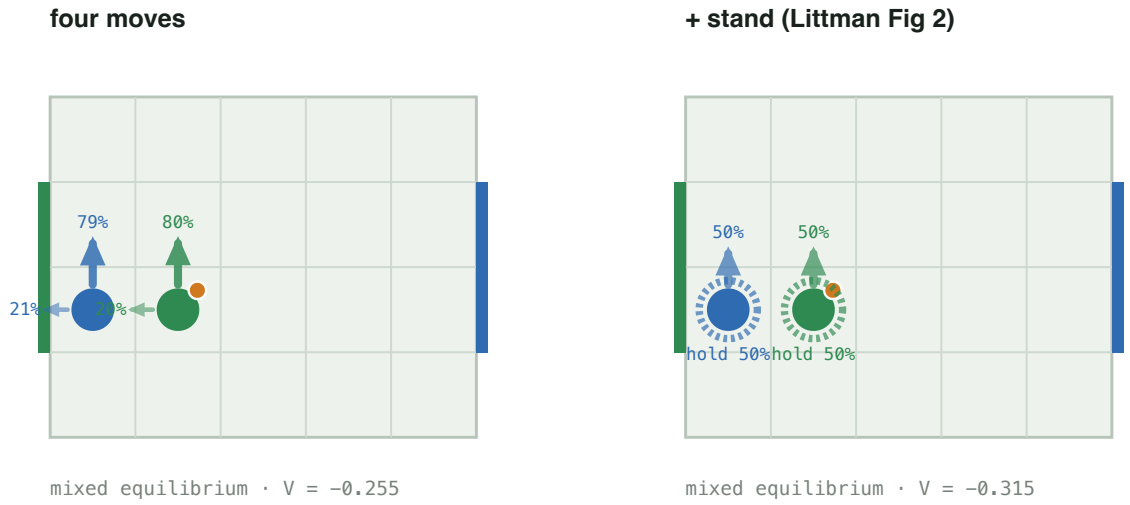
Littman's fifth action

Littman's 1994 soccer game has a fifth action, `stand`, and his Figure 2 — a carrier pinned against its own goal by an adjacent defender — is the textbook "soccer needs mixing" example: the carrier must randomize between standing and moving. Adding `stand` to the random-order game (`n_actions=5`, `scripts/littman.py`) on Littman's 5×4 board:

LITTMAN'S BOARD (5×4, 2-CELL GOALS), RANDOM MOVE ORDER, DISCOUNT 0.9

ACTION SET	NO-SADDLE STATES	STAND IN THE MIXED SUPPORT	V(KICKOFF)
N, S, E, W	56	0	+0.147
N, S, E, W, stand	56	48	+0.172

The fifth action does *not* move where mixing happens — 48 of the 56 no-pure-saddle states are shared, 8 swap in and 8 out at the margin — but it changes *what* the mix is: the carrier now hedges with `stand` in 48 of 56 states rather than with `retreat`, and every player is slightly better for the option (`V(kickoff) +0.147 → +0.172`). The Figure 2 state is a clean matching-pennies matrix with equilibrium (`½ climb, ½ hold`). The deterministic game keeps a pure saddle at every state with five actions too.

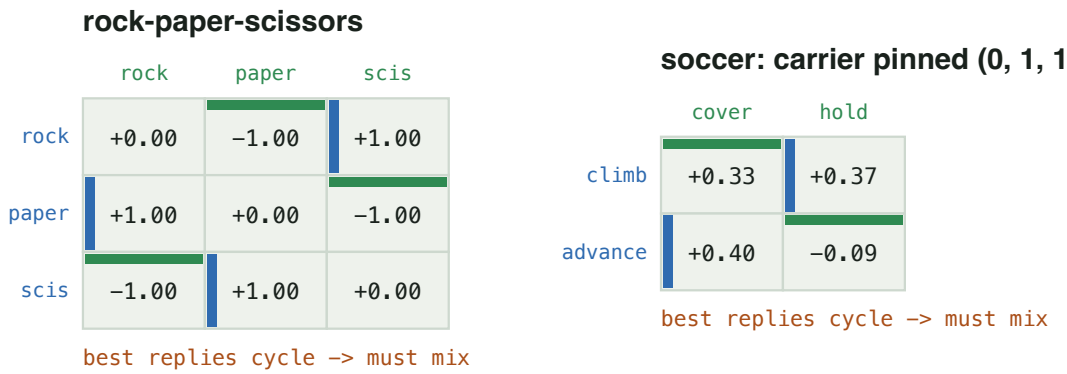


Littman's Figure 2. Same state, four moves (left) vs. five (right). The dashed ring is `stand`. Adding it turns the carrier's climb/retreat hedge into climb/hold — Littman's mixed policy.

RQ4 — Numerical robustness

The soccer game contains rock-paper-scissors

The pure/mixed check is the professor's $O(A^2)$ best-reply method: mark player 0's best row in each column and player 1's best column in each row; a cell with both is a pure saddle, and if none has both the best replies *cycle* and the game needs mixing — no LP required.



Same structure. Left: rock-paper-scissors. Right: the stage game at (0, 1, 1, 1, 1) — carrier pinned against its own goal, defender adjacent (Littman's Figure 2 geometry). Blue bar = player 0's best row per column; green bar = player 1's best column per row. They cross both ways — matching pennies, no pure equilibrium.

The value is three numbers

`value_bracket` returns what the row player guarantees ($\min_k (pM)_k$), gets ($p^T Mq$), and can reach ($\max_i (Mq)_i$) — the three quantities the professor flagged as never numerically identical. The true value is bracketed, so the width certifies the LP error.

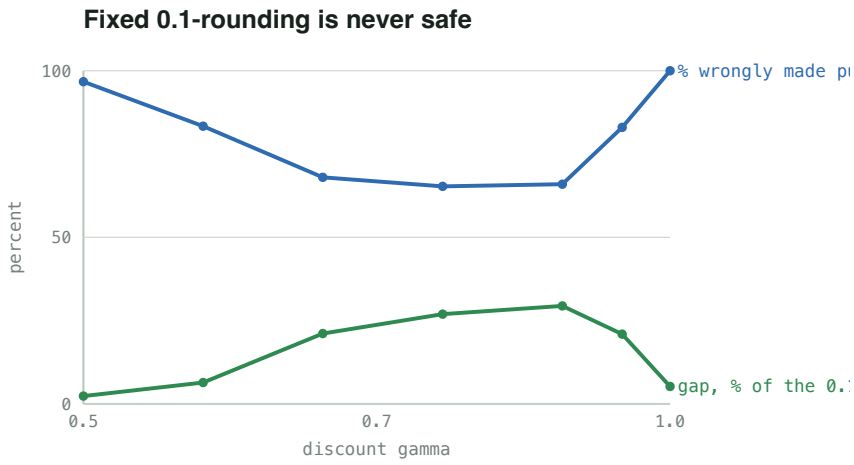
FINDING

With `scipy`'s HiGHS the three agree to `1.1e-16` per stage game, `8.7e-10` accumulated — the definitional ambiguity is not a practical problem. If it were (an older vertex solver, or accumulated drift), the row player should take the *guaranteed* value $\min_k (pM)_k$: it is always a safe lower bound on the true minimax.

Discounting and tolerance

	DETERMINISTIC	RANDOM (7×5)
mixed states, γ = 0.5 → 0.9 → 0.995	0 → 0 → 0	122 → 94 → 102
classification split, rel_tol 1e−12 → 1e−3	stable	604 / 1682 / 94 (stable)
classification, rel_tol 1e−2 / 1e−1	—	80 / 0 mixed
fixed 0.1-rounding flips a saddle	0	62

Discounting mildly shifts which states mix; the classification is otherwise robust across nine orders of magnitude of tolerance, breaking only as the tolerance approaches the real `minimax − maximin` gaps. Fixed 0.1-rounding — proposed to force indifference — misclassifies 62 states, exactly the small-entry mixed region.



There is no safe discount. The genuine `minimax − maximin` gaps (green) never reach the 0.1 rounding grid at any γ: heavy discounting shrinks the γ^k value differences, light discounting leaves entries barely apart. Fixed 0.1-rounding wrongly collapses 60–100% of the mixed games (blue). The scale-aware `rel_tol` classifier is stable across the whole range.

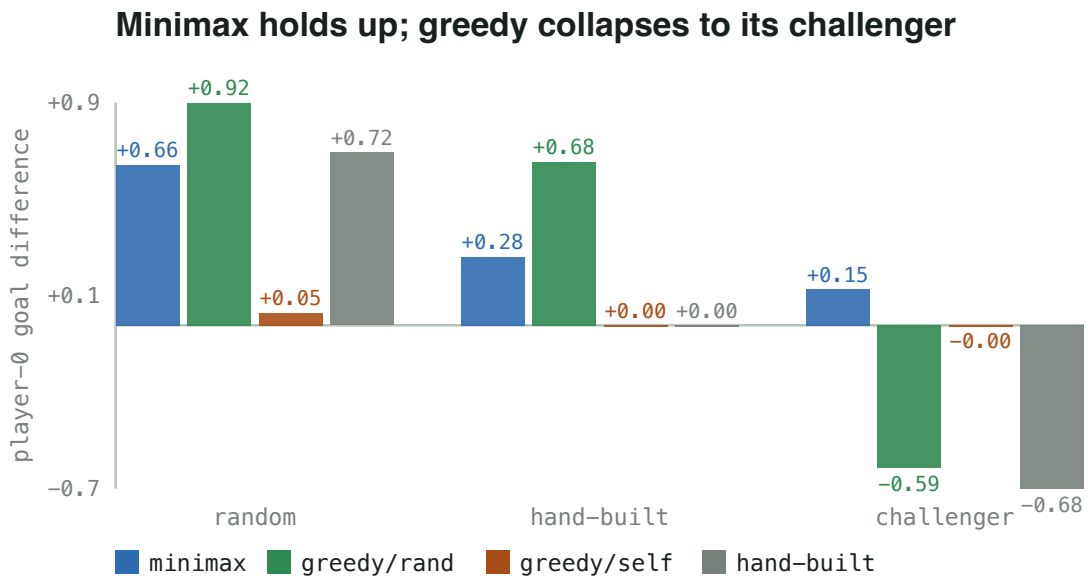
Supporting evidence — the policy plays correctly

Littman's Table 3: minimax exploits *and* survives

Littman's central experiment trains four policies and scores each against a random opponent, a hand-built one ("simple rules for scoring and blocking", §6.2), and a **challenger** trained to beat it — his minimax policies held ~35–40% of games against their challenger, his Q-learning (deterministic) policies won **0%**. Reproduced here with the *exact* solver on his 5×4 / 5-action / random-order / goal-reset board (`scripts/tournament.py`, score = player-0 expected discounted goal difference; the hand-built opponent beats a random one ~76% of games):

LITTMAN'S BOARD, Γ = 0.9 — PLAYER-0 GOAL DIFFERENCE FROM THE KICKOFF

PLAYER-0 POLICY	VS. RANDOM	VS. HAND-BUILT	VS. ITS CHALLENGER	ROBUSTNESS GAP
minimax (Nash)	+0.67	+0.28	**+0.15**	+0.52
greedy vs. random	+0.92	+0.68	**−0.59**	+1.51
greedy self-play	+0.05	0.00	−0.00	+0.05
hand-built	+0.72	0.00	**−0.68**	+1.39



Only minimax is in the top-left. greedy/rand is the strongest policy against weak opponents and among the weakest against a challenger — the rock-paper-scissors trap. minimax presses its advantage (+0.67) and still wins (+0.15) against the worst case, because the Nash policy has no exploitable row.

The mechanism is exactly Littman's "every deterministic offense has a perfect defense": the mixed states are matching-pennies stage games; the greedy *and* the hand-built policy each commit to one row of each, and the challenger plays the column that beats it. The minimax policy randomizes with the weights that make the challenger indifferent. Full table (including the `P(win) − P(loss)` view) in `docs/tournament.md`.

Other checks

- Duality gap `V0_br(s₀) + V1_br(s₀) < 1e−9` for every move order — no opponent beats the game value.
- Nash vs. Nash reproduces the value: forced draws in the deterministic and coin-flip games; `+0.162 ± 0.002` empirical discounted return over 5 seeds of 3000 games (vs. `+0.164` computed) in the random game.
- A best response to one assumed opponent is fragile: the Part 2 policy has exploitability `0.43` — worse than random — which is the A10 competition's warning about non-equilibrium submissions.
- A10 networks over 5 seeds (`experiments/a10_competition_seeds.csv`): the Part 2 imitation net plays optimally at `0.990 ± 0.000` of states and wins Q8 every time; competition Network Second is exact, but Network First sits at a `0.19 ± 0.02` exploitability from this kickoff that does not wash out with re-seeding.

Exploratory — function approximation and general-sum

These are groundwork for the eventual continuous-action work, not equal-weight contributions. The exact solver is the point of both: it is the ground truth they are measured against.

Two neural baselines, and where the approximation breaks

On the deterministic game (5 seeds, `experiments/nash_dqn_seeds.csv`): a **Q net** regresses toward the stage matrices $Q(s, a_0, a_1)$ and extracts a policy by minimax; a **policy net** regresses *directly* onto the exact equilibrium strategies.

5 SEEDS	EXACT	Q NET (MEAN ± SD)	POLICY NET
max |V − V_exact|	0	0.55 ± 0.02	—
action agreement	100%	43% ± 2%	99.9% ± 0.0%
pure/mixed classification	100%	67% ± 2%	—
max equilibrium regret ε_{NE}	0	—	0.46 ± 0.02
exploitability (duality gap)	<1e−9	0.44 ± 0.05	0.84 ± 0.00

The policy net names the exact-optimal action at 99.9% of states and is *more* exploitable than the mediocre Q net (duality gap 0.84 vs 0.44). Two reasons: the ~0.1% of states where it is wrong are exactly the ones a best-responder attacks (the rock-paper-scissors trap again), and a softmax head necessarily smears what should be pure strategies, manufacturing exploitable near-indifference. So the failure is not primarily value approximation — it is that "name the right action" is not the same as "play a Nash". Even on a small discrete game where the exact solution is a 0.3 s computation, a straightforward neural approximation does not preserve game-theoretic robustness. Keep the exact solver as ground truth.

General-sum — a sanity check, not a second paper

`markov_game.py` extends pure-first beyond zero-sum: enumerate *all* stage equilibria (`support_enum.py`), select one by "largest sum of values". On Battle of the Sexes this correctly avoids the mixed equilibrium whose value is below either pure one; the soccer game is zero-sum, so it does not touch the main result. Left as future work.

Limitations

- The A10 page redacts the ID-specific start position and goal rows; this repo uses the standard Littman geometry (configurable). Across the phase-diagram sweep every one-cell-goal board has 0 mixed states and every wider-goal board has some; the exact `3.95%` is the 7×5 / 3-cell-goal case.
- Several collision sub-cases (carrier vs. stationary opponent, bump = steal, swap possession) are *interpreted* from the two stated A10 rules, not quoted. If course staff intended otherwise, Claim A must be re-measured. See `docs/assumptions.md`.
- **Claim A/B** (a pure equilibrium of the deterministic game) is established constructively — the explicit pure memoryless attractor/safety profile. **Claim C** (the theoretical game necessarily has one, over all rules and settings) is *not*, for the general goal mouth; for the single goal cell it is partly proved (`docs/proof.md`).
- `random` and `coinflip` are different game definitions, isolating the collision rule and the tie-break respectively — not the A10 game.
- The goal-width result is **empirical over the tested grid**: a multi-cell goal is necessary for the mixed region in the studied family and sufficient for at least one mixed state on the tested boards — not a theorem.

What weight each experiment carries

Core. Pure vs. mixed by collision rule (RQ1); the pure-first hybrid, exact, LP eliminated on 96–100% (RQ2); the phase diagram — goal width, not board size (RQ3); geometry → templates → 2×2 matching pennies (RQ3); numerical robustness — the three value numbers, the rounding trap, the classifier (RQ4).

Strong supporting evidence. Occupancy (4% of states, 41% of the path); Littman Table 3 reproduced; the reward-objective comparison (region invariant, value range not); symmetry (value *and* policy equivariance); the `blend` interpolation.

Future / exploratory. The two neural baselines; the general-sum extension; freeze-then-iterate and the `eval_value` choice; the single-cell dominance / attractor machinery.

Related work

- **A. Markov games and minimax-Q.** Littman (1994) — the two-player zero-sum Markov game, minimax by LP, soccer *because* its optimal policy can be probabilistic. The baseline this project builds on.
- **B. Concurrent / reachability games.** de Alfaro–Henzinger–Kupferman on positional determinacy — the frame for the attractor result and the single-cell argument.
- **C. Stochastic games and stationary equilibria.** Shapley (1953); Filar & Vrieze; the orderfield-property literature — the frame for "one target cell vs. many".
- **D. Strategy improvement.** Hoffman–Karp, Condon for turn-based games; the *absence* of a monotone guarantee for concurrent games explains the freeze-then-iterate thrash.
- **E. Reward shaping.** Ng–Harada–Russell potential-based shaping — the frame for the PBRS-leaves-the-equilibrium-exact result.

Prior work: mixed strategies can be necessary in soccer (A); pure-first enumeration before LP (the meeting). *This project adds:* an empirical map of where the mixed region is, what switches it on, how much of the equilibrium path it covers, and a hybrid solver that pays LP cost only there. `docs/result.md` §6 positions the goal-width switch against Littman (1994), the ordered-field property, concurrent reachability games, and pursuit–evasion in full.

`soccer_nash — game.py · matrix_games.py · support_enum.py · nash_q.py · markov_game.py · numerics.py · dominance.py · attractor.py · geometry.py · symmetry.py · tree.py · templates.py · onecell.py · certificate.py · reachability.py · occupancy.py · evaluate.py · best_response.py · exploit.py · mlp.py · nash_dqn.py · opponents.py · shaping.py · simulate.py · experiment.py · render.py · viz.py`
`scripts — experiments.py · analyze.py · numerics.py · equilibria.py · geometry_model.py · templates.py · phase_diagram.py · onecell_proof.py · verify_mechanism.py · generalize.py · tackle.py · positional.py · benchmark.py · policy_iteration.py · selfplay.py · nash_dqn.py · render.py · gallery.py · story.py · littman.py · mixing.py · reward.py · blend.py · occupancy.py · tournament.py · a10_part1.py · a10_part2.py · a10_competition.py`
`docs — design.md · advisor.md · methods.md · discussion.md · numerics.md · assumptions.md · templates.md · proof.md · geometry.md · mechanism.md · result.md · generalize.md · tackle.md · littman.md · tournament.md · mixing.md · reward.md · blend.md · occupancy.md · gallery.html`
`experiments — baseline.csv · undiscounted.csv · gamma_sweep.csv · tolerance_sweep.csv · board_sweep.csv · goal_mouth_sweep.csv · one_cell_goal.csv · phase_diagram.csv · nash_dqn_seeds.csv`
`env: 7×5 grid, 2 380 states, zero-sum ±1, γ = 0.9`